

First-Time Setup (Initialization) Procedure — v0.9 DRAFT

15 July 2026 · Drafted from the lessons of Sessions 1–5 — most of which were first-time-setup failures this procedure exists to prevent · Run ONCE per matter or workstation · Companion documents: Wake-Up Procedure (every session thereafter); Sleep Procedure (to be drafted); Pilot Synthesis §3 (doctrine) · Status: DRAFT — to be reviewed in Session 6 and issued v1.0.

1. Purpose

This procedure takes a workstation from nothing to a verified, documented starting state — the "session zero" baseline that every subsequent wake-up verifies against. Initialization CREATES the known state; the Wake-Up Procedure only CONFIRMS it. The pilot's own Session 1 was an undocumented initialization, and its failures — an over-broad trust grant that enabled a contamination two days later, a primer paste that silently never landed — are exactly the class this procedure prevents. The principle throughout: decisions made once at setup (trust boundaries, folder geometry, window discipline) silently govern everything after; make them deliberately, record them, and never let a default make them for you.

2. The physical workstation

Recommendation, in order of preference. FIRST: a dedicated machine for evidence work — nothing else installed or signed in, no personal email, no cloud sync agents (OneDrive, Dropbox, Google Drive) touching the evidence drive, no other AI tools. The pilot found the tool writing to at least six locations under the user profile without asking; on a shared machine those records commingle with everything else the user does, and sync agents can quietly ship evidence fragments to third-party clouds. SECOND BEST: a dedicated Windows user account on a shared machine — the tool's stores live under the user profile, so a dedicated account at least fences them. MINIMUM: the pilot's own configuration — a general-purpose machine with a dedicated evidence drive — accepted with recorded caveats: tool stores commingle on C:, and the operator must know it.

Hardware notes from the pilot: the local machine does no thinking — CPU/memory load is negligible; the bottleneck is disk I/O on hundreds of thousands of small files (whole-corpus size sums ran 30+ minutes and sometimes never finished). An SSD for the evidence drive matters more than processor or RAM. Ample free space on C: is required regardless, because the tool's own stores (transcripts, file-history checkpoints, backups) grow with every session. Reliable network is required at all times — all inference is cloud-side; the honest posture name is "workstation-mediated with logged custody, cloud inference", and if data may not leave the building, this architecture is the wrong one (a local-model variant is a different project).

Windows hygiene at setup: show file extensions and hidden files in Explorer (the pilot's .git and double-extension findings were invisible without this); note that the machine will be left running unattended between sessions and the tool updates itself when unattended — there is no current way to prevent this, so the wake-up ritual checks versions every session; disable any backup/sync software's coverage of the evidence drive, or document precisely what touches it, because every silent copier is a custody actor.

3. Folder geometry — the decision that governs everything

One decision at setup determines most of the pilot's later safety: the trust boundary. Claude Code, when launched, asks to trust the folder it is launched from, and that grant covers everything beneath it AND DOES NOT EXPIRE. Therefore the geometry rule: the WORKSPACE (what the tool may see) is one folder;

everything the tool must never see — exhibits, banked primers, session logs, procedures, audit transcripts — lives one level ABOVE it, outside the boundary forever. Evidence sits INSIDE the workspace as a protected read-only object, never AS the workspace. Reference layout (the pilot's, generalise names as needed):

```
E:\<Matter>\                                <- pilot root: NEVER launch the tool here
  exhibits, logs, procedures, banked primers  <- above the boundary, invisible to the
subject
  <workspace>\                                <- the ONLY folder ever trusted or launched
from
  CLAUDE.md                                <- the deployed primer
  <evidence>\                                <- read-only corpus, inside the workspace
  pilot_outputs\                            <- the subject's only writable destination
```

4. The procedure

Step 1 — Open the Session Zero log

Before anything is installed or created: open the pilot log's first entry. Record machine identity, Windows build, drive layout, who has accounts on the machine, what sync/backup software exists, and every decision this procedure makes as it is made. This entry IS the baseline every future wake-up verifies against.

Done when: A written record exists and is being filled as setup proceeds.

Step 2 — Prepare the physical workstation

Apply Section 2: dedicated machine or account if possible; extensions and hidden files visible; sync agents excluded from the evidence drive (verify, don't assume); free space checked on both drives. Record the configuration achieved, including accepted compromises — an honest caveat recorded now beats a silent assumption discovered in week three.

Done when: Workstation state recorded, compromises named.

Step 3 — Create the folder geometry

Build the Section 3 layout empty, top-down: matter root, exhibit level, workspace, evidence folder, outputs folder. Do this BEFORE any evidence arrives and BEFORE the tool is installed, so no launch can ever happen against an unplanned geometry.

```
mkdir "E:\<Matter>"
mkdir "E:\<Matter>\<workspace>"
mkdir "E:\<Matter>\<workspace>\pilot_outputs"
```

Done when: Geometry exists and matches the recorded plan.

Step 4 — Ingest the evidence and freeze its identity

Copy the corpus into the evidence folder. Then record what arrived: file count, total logical size, and — if time permits — a content-hash manifest (even a partial one anchors integrity claims later). Record the provenance of the copy itself: where it came from, what transformations it has undergone (the pilot spent a day reconstructing an undocumented bulk rename; ten minutes of notes at ingest would have prevented it). Set the evidence folder read-only at the filesystem level as a belt alongside the primer's braces.

```
(Get-ChildItem "E:\<Matter>\<workspace>\<evidence>" -Recurse -File | Measure-Object -
Property Length -Sum) | Select-Object Count, Sum
```

Done when: Count, size, and provenance notes in the log; any pre-existing manifest preserved unmodified beside the evidence.

Step 5 — Create and verify the primer

Write CLAUDE.md into the workspace — via Set-Content or careful editor use, never a paste whose destination you haven't confirmed (Session 1's paste landed in the terminal and the wrong primer governed for a day). Verify by SIZE first, CONTENTS second, read against the master copy word for word — impression is not verification. Bank the master copy at the exhibit level; the workspace copy is a deployment.

```
type "E:\<Matter>\<workspace>\CLAUDE.md"
dir "E:\<Matter>\<workspace>\CLAUDE.md"
```

Done when: Byte size and full text match the banked master.

Step 6 — Install the tool and record its store anatomy

Install Claude Code (per current official instructions — they change; do not trust a stale note). Then, BEFORE first launch, census the tool's home under the user profile and record what exists: this is the store baseline. The pilot found, over five sessions: projects (transcripts, tool-results), sessions, file-history (per-session file checkpoints), backups (config snapshots), paste-cache, session-env — and the anatomy MOVED between versions. Whatever exists on your install date is your baseline; every wake-up diffs against it.

```
dir "C:\Users\<user>\.claude" -Directory
```

Done when: Anatomy recorded in the Session Zero log, with the tool version that produced it.

Step 7 — Establish the two-window discipline physically

Create the AUDIT BENCH: a fresh PowerShell window, retitled, self-transcribing, parked at the matter root — never inside the workspace. Designate a second window as the subject channel. The rule is physical, not mental: tasks go only in the window with the Claude prompt; everything else goes only in the window titled AUDIT BENCH; a window that is neither gets closed. (Every routing failure of the pilot traces to windows whose role was ambiguous.)

```
$Host.UI.RawUI.WindowTitle = "AUDIT BENCH - do not type tasks here"
Start-Transcript "E:\<Matter>\audit-bench-transcript-session0.txt"
```

Done when: Two windows, visually distinct, roles recorded.

Step 8 — First launch: grant trust narrowly and knowingly

From the subject's window, cd into the WORKSPACE — read the path aloud — and launch. The trust prompt that appears is the single most consequential click of the setup: it grants the tool read/edit/execute over that folder, forever, without further warning. Confirm the path shown is the workspace and nothing broader, then trust. If the path shows anything else, exit and fix — the pilot's Session 1 trusted a folder containing the entire experiment's documentation, and the grant was still enabling mistakes two days later. Record the banner: client version, model. This is the subject's birth certificate.

```
cd "E:\<Matter>\<workspace>"
claude
```

Done when: Trust granted to the workspace path only; version and model recorded.

Step 9 — Verify the primer loaded, then close cleanly

Ask the subject what governs it: a correct answer comes from loaded context, citing the primer, with no shell commands. Then exit, note any resume ID the banner prints, and confirm the tool's store gained

exactly one session's records. Initialization does NOT proceed to real work — it ends with a verified clean state and a complete Session Zero log. Real work begins with the Wake-Up Procedure, next session, verifying against everything recorded today.

```
What primer governs this session?  
/exit
```

Done when: Primer confirmed from context; session closed; store delta recorded; Session Zero log complete and saved at the exhibit level.

5. What this procedure cannot control (recorded honestly)

The tool updates itself unattended and cannot currently be pinned — versions are recorded at every launch and drift is treated as environment (subject to the Session 6 frozen-subject investigation). Trust grants persist invisibly and cannot be audited from the prompt — the geometry rule is the defence. The tool's stores on C: will accumulate evidence-derived fragments (transcripts, tool-results, checkpoints) — on any sensitive matter, those stores are themselves evidence-bearing and their end-of-matter disposal must be planned at setup, not discovered at closure. And inference is cloud-side: content read by the subject leaves the building by design; if that is unacceptable for the matter, stop at Step 2 and choose a different architecture.